Python Engineering

Class 08 - Data Processing | EX16b - Masks & Filtering Practice

No week or class change. We remain in **week_02_data_structures -> class_08_data_processing**. This exercise is designed to automate condition -> mask -> filter -> result reasoning.

Goal

Use a small fixed dataset to practice boolean masks, condition combinations, counts, indices, row filtering, statistics, derived columns, and time checks without the extra complexity of Euler integration.

Dataset

```
# columns: [time, altitude, velocity, temperature]
sensor_data = np.array([
    [0.0,  20.0,  30.0, 18.0],
    [1.0,  55.0,  25.0, 17.0],
    [2.0,  90.0,  18.0, 15.0],
    [3.0, 120.0,  10.0, 13.0],
    [4.0, 140.0,   2.0, 11.0],
    [5.0, 138.0,  -6.0, 10.0],
    [6.0, 115.0, -15.0, 12.0],
    [7.0,  80.0, -22.0, 14.0],
    [8.0,  40.0, -28.0, 16.0]
])
```

BLOCK A - Extract columns

```
times = ...
altitudes = ...
velocities = ...
temperatures = ...

# Print shape of every extracted vector.
```

BLOCK B - Simple masks

```
Create masks for:
- altitude >= 100 m
- velocity > 0 m/s
- velocity < 0 m/s
- temperature < 15 degC
```

For each mask print the mask, number of True values with np.sum(mask), and indices with np.where(mask)[0].

BLOCK C - Filter values vs rows

```
# Using the altitude mask:
high_altitude_values = ...
high_altitude_states = ...

# Compare their shapes.
```

`altitudes[mask]` filters values from one column; `sensor_data[mask]` filters complete rows.

BLOCK D - Combine conditions

```
Create and filter:
1. altitude >= 100 AND velocity > 0
2. altitude >= 100 OR temperature < 12
3. NOT ascending
4. time between 2 and 6 seconds inclusive
```

BLOCK E - Filtered statistics

```
# For states with altitude >= 100 m:
- mean altitude
```

```
- std altitude
- mean velocity
- minimum temperature
```

BLOCK F - Derived variable

```
speed = np.abs(velocities)

processed_data = np.column_stack((
    sensor_data,
    speed
))

# [time, altitude, velocity, temperature, speed]
```

`np.abs()` receives a number/array and returns element-wise absolute values.

BLOCK G - Sampling time

```
dt_samples = np.diff(times)
constant_dt = np.allclose(dt_samples, 1.0)
```

BONUS - np.any() and np.all()

np.any(mask) receives a boolean array and returns True if at least one element is True. **np.all(mask)** returns True only if every element is True.

```
any_above_130 = np.any(altitudes > 130)
all_temperatures_positive = np.all(temperatures > 0)
```

FINAL CHECK

```
Explain in comments:
- Difference between x[mask] and A[mask]
- Difference between np.sum(mask) and np.where(mask)[0]
- Difference between & and |
- What ~mask does
- What np.any() returns
- What np.all() returns
- Why np.diff(times) has one element less than times
```

Completion criterion

EX16b is complete when you can read a filtering question and decide which column is needed, which condition creates the mask, whether you want values or rows, and whether you need a count, indices, or a statistic.